Rebuilding Paragraph Structure in the Browser

[0] The reconstruction of paragraph structure from a positioned text stream is the central difficulty in porting a desktop reader to the browser. PyMuPDF exposes ready-made blocks, whereas PDF.js returns only runs with transforms. Consequently the grouping heuristic must infer lines from vertical proximity and columns from horizontal gutters before any merging can take place. The reconstruction of paragraph structure from a positioned text stream is the central difficulty in porting a desktop reader to the browser. PyMuPDF exposes ready-made blocks, whereas PDF.js returns only runs with transforms. Consequently the grouping heuristic must infer lines from vertical proximity and columns from horizontal gutters before any merging can take place. The reconstruction of paragraph structure from a positioned text stream is the central difficulty in porting a desktop reader to the browser. PyMuPDF exposes ready-made blocks, whereas PDF.js returns only runs with transforms. Consequently the grouping heuristic must infer lines from vertical proximity and columns from horizontal gutters before any merging can take place.

[1] The reconstruction of paragraph structure from a positioned text stream is the central difficulty in porting a desktop reader to the browser. PyMuPDF exposes ready-made blocks, whereas PDF.js returns only runs with transforms. Consequently the grouping heuristic must infer lines from vertical proximity and columns from horizontal gutters before any merging can take place. The reconstruction of paragraph structure from a positioned text stream is the central difficulty in porting a desktop reader to the browser. PyMuPDF exposes ready-made blocks, whereas PDF.js returns only runs with transforms. Consequently the grouping heuristic must infer lines from vertical proximity and columns from horizontal gutters before any merging can take place.

[2] The reconstruction of paragraph structure from a positioned text stream is the central difficulty in porting a desktop reader to the browser. PyMuPDF exposes ready-made blocks, whereas PDF.js returns only runs with transforms. Consequently the grouping heuristic must infer lines from vertical proximity and columns from horizontal gutters before any merging can take place. The reconstruction of paragraph structure from a positioned text stream is the central difficulty in porting a desktop reader to the browser. PyMuPDF exposes ready-made blocks, whereas PDF.js returns only runs with transforms. Consequently the grouping heuristic must infer lines from vertical proximity and columns from horizontal gutters before any merging can take place.

[3] The reconstruction of paragraph structure from a positioned text stream is the central difficulty in porting a desktop reader to the browser. PyMuPDF exposes ready-made blocks, whereas PDF.js returns only runs with transforms. Consequently the grouping heuristic must infer lines from vertical proximity and columns from horizontal gutters before any merging can take place. The reconstruction of paragraph structure from a positioned text stream is the central difficulty in porting a desktop reader to the browser. PyMuPDF exposes ready-made blocks, whereas PDF.js returns only runs with transforms. Consequently the grouping heuristic must infer lines from vertical proximity and columns from horizontal gutters before any merging can take place.

[4] The reconstruction of paragraph structure from a positioned text stream is the central difficulty in porting a desktop reader to the browser. PyMuPDF exposes ready-made blocks, whereas PDF.js returns only runs with transforms. Consequently the grouping heuristic must infer lines from vertical proximity and columns from horizontal gutters before any merging can take place. The reconstruction of paragraph structure from a positioned text stream is the central difficulty in porting a desktop reader to the browser. PyMuPDF exposes ready-made blocks, whereas PDF.js returns only runs with transforms. Consequently the grouping heuristic must infer lines from vertical proximity and columns from horizontal gutters before any merging can take place.

Method

[5] The reconstruction of paragraph structure from a positioned text stream is the central difficulty in porting a desktop reader to the browser. PyMuPDF exposes ready-made blocks, whereas PDF.js returns only runs with transforms. Consequently the grouping heuristic must infer lines from vertical proximity and columns from horizontal gutters before any merging can take place. The reconstruction of paragraph structure from a positioned text stream is the central difficulty in porting a desktop reader to the browser. PyMuPDF exposes ready-made blocks, whereas PDF.js returns only runs with transforms. Consequently the grouping heuristic must infer lines from vertical proximity and columns from horizontal gutters before any merging can take place.

[6] The reconstruction of paragraph structure from a positioned text stream is the central difficulty in porting a desktop reader to the browser. PyMuPDF exposes ready-made blocks, whereas PDF.js returns only runs with transforms. Consequently the grouping heuristic

must infer lines from vertical proximity and columns from horizontal gutters before any merging can take place. The reconstruction of paragraph structure from a positioned text stream is the central difficulty in porting a desktop reader to the browser. PyMuPDF exposes ready-made blocks, whereas PDF.js returns only runs with transforms. Consequently the grouping heuristic must infer lines from vertical proximity and columns from horizontal gutters before any merging can take place. The reconstruction of paragraph structure from a positioned text stream is the central difficulty in porting a desktop reader to the browser. PyMuPDF exposes ready-made blocks, whereas PDF.js returns only runs with transforms. Consequently the grouping heuristic must infer lines from vertical proximity and columns from horizontal gutters before any merging can take place.

[7] The reconstruction of paragraph structure from a positioned text stream is the central difficulty in porting a desktop reader to the browser. PyMuPDF exposes ready-made blocks, whereas PDF.js returns only runs with transforms. Consequently the grouping heuristic must infer lines from vertical proximity and columns from horizontal gutters before any merging can take place. The reconstruction of paragraph structure from a positioned text stream is the central difficulty in porting a desktop reader to the browser. PyMuPDF exposes ready-made blocks, whereas PDF.js returns only runs with transforms. Consequently the grouping heuristic must infer lines from vertical proximity and columns from horizontal gutters before any merging can take place.

[8] The reconstruction of paragraph structure from a positioned text stream is the central difficulty in porting a desktop reader to the browser. PyMuPDF exposes ready-made blocks, whereas PDF.js returns only runs with transforms. Consequently the grouping heuristic must infer lines from vertical proximity and columns from horizontal gutters before any merging can take place. The reconstruction of paragraph structure from a positioned text stream is the central difficulty in porting a desktop reader to the browser. PyMuPDF exposes ready-made blocks, whereas PDF.js returns only runs with transforms. Consequently the grouping heuristic must infer lines from vertical proximity and columns from horizontal gutters before any merging can take place.

[9] The reconstruction of paragraph structure from a positioned text stream is the central difficulty in porting a desktop reader to the browser. PyMuPDF exposes ready-made blocks, whereas PDF.js returns only runs with transforms. Consequently the grouping heuristic must infer lines from vertical proximity and columns from horizontal gutters before any merging can take place. The reconstruction of paragraph structure from a positioned text stream is the central difficulty in porting a desktop reader to the browser. PyMuPDF exposes ready-made blocks, whereas PDF.js returns only runs with transforms. Consequently the grouping heuristic must infer lines from vertical proximity and columns from horizontal gutters before any merging can take place. The reconstruction of paragraph structure from a positioned text stream is the central difficulty in porting a desktop reader to the browser. PyMuPDF exposes ready-made blocks, whereas PDF.js returns only runs with transforms. Consequently the grouping heuristic must infer lines from vertical proximity and columns from horizontal gutters before any merging can take place.

[10] The reconstruction of paragraph structure from a positioned text stream is the central difficulty in porting a desktop reader to the browser. PyMuPDF exposes ready-made blocks, whereas PDF.js returns only runs with transforms. Consequently the grouping heuristic must infer lines from vertical proximity and columns from horizontal gutters before any merging can take place. The reconstruction of paragraph structure from a positioned text stream is the central difficulty in porting a desktop reader to the browser. PyMuPDF exposes ready-made blocks, whereas PDF.js returns only runs with transforms. Consequently the grouping heuristic must infer lines from vertical proximity and columns from horizontal gutters before any merging can take place.

[11] The reconstruction of paragraph structure from a positioned text stream is the central difficulty in porting a desktop reader to the browser. PyMuPDF exposes ready-made blocks, whereas PDF.js returns only runs with transforms. Consequently the grouping heuristic must infer lines from vertical proximity and columns from horizontal gutters before any merging can take place. The reconstruction of paragraph structure from a positioned text stream is the central difficulty in porting a desktop reader to the browser. PyMuPDF exposes ready-made blocks, whereas PDF.js returns only runs with transforms. Consequently the grouping heuristic must infer lines from vertical proximity and columns from horizontal gutters before any merging can take place.

[12] The reconstruction of paragraph structure from a positioned text stream is the central difficulty in porting a desktop reader to the browser. PyMuPDF exposes ready-made blocks, whereas PDF.js returns only runs with transforms. Consequently the grouping heuristic must infer lines from vertical proximity and columns from horizontal gutters before any merging can take place. The reconstruction of paragraph structure from a positioned text stream is the central difficulty in porting a desktop reader to the browser. PyMuPDF exposes ready-made blocks, whereas PDF.js returns only runs with transforms. Consequently the grouping heuristic must infer lines from vertical proximity and columns from horizontal gutters before any merging can take place. The reconstruction of paragraph structure from a positioned text stream is the central difficulty in porting a desktop reader to the browser.

PyMuPDF exposes ready-made blocks, whereas PDF.js returns only runs with transforms. Consequently the grouping heuristic must infer lines from vertical proximity and columns from horizontal gutters before any merging can take place.

[13] The reconstruction of paragraph structure from a positioned text stream is the central difficulty in porting a desktop reader to the browser. PyMuPDF exposes ready-made blocks, whereas PDF.js returns only runs with transforms. Consequently the grouping heuristic must infer lines from vertical proximity and columns from horizontal gutters before any merging can take place. The reconstruction of paragraph structure from a positioned text stream is the central difficulty in porting a desktop reader to the browser. PyMuPDF exposes ready-made blocks, whereas PDF.js returns only runs with transforms. Consequently the grouping heuristic must infer lines from vertical proximity and columns from horizontal gutters before any merging can take place.

A unique needle phrase quixotic zephyr appears exactly once.